

Evaluationskapitel: Validierung von n8n-Workflows

Ziel

Ziel der Evaluation ist es, verschiedene Validierungsansätze für n8n-Workflows systematisch zu vergleichen. Es soll untersucht werden, welche Verfahren fehlerhafte Workflows zuverlässig erkennen, ab welcher Komplexität Fehler übersehen werden und ob sich für LLM-Aufgaben innerhalb eines Workflows auch nicht-LLM-basierte Prüfverfahren eignen.

Grundidee

Jeder Testfall besteht aus:

- einer fachlichen Beschreibung
- einem korrekten Referenz-Workflow
- mehreren absichtlich fehlerhaften Varianten
- einem oder mehreren Validatoren

Für jeden Validator wird gemessen, ob er korrekte und inkorrekte Workflows richtig klassifiziert und welche Fehlerarten er erkennt oder übersieht.

Datensatzaufbau

Ein realistischer Datensatz kann aus 100 Basis-Workflows bestehen.

Empfohlene Verteilung:

- 40 klassische sequenzielle Business-Workflows
- 30 Workflows mit Branching und Parallelität
- 20 komplexere Ausnahme- und Freigabeprozesse
- 10 Workflows mit 1-2 LLM-Knoten

Zu jedem Basis-Workflow werden mehrere Varianten erzeugt:

- 1 korrekter Workflow
- 1 Strukturfehler-Variante
- 1 Kontrollflussfehler-Variante
- 1 Semantikfehler-Variante
- bei LLM-Workflows zusätzlich 1 LLM-Fehler-Variante

Dadurch entstehen etwa 410 bis 500 Testobjekte.

Fehlerklassen

Strukturfehler

- fehlender Node
- falsche Connection
- toter Pfad

- nicht erreichbarer Node
- fehlender Merge

Kontrollflussfehler

- vertauschter IF-Branch
- fehlende Parallelität
- falsche Reihenfolge
- falsch platzierter Wait- oder Retry-Schritt

Semantikfehler

- fachlich falscher Schritt
- fehlender Muss-Schritt trotz formal gültigem Workflow
- falscher Empfänger einer Nachricht
- Rechnung wird vor Freigabe gebucht

LLM-bezogene Fehler

- LLM-Node an falscher Position
- LLM-Node mit falschem Zweck
- LLM-Ausgabe wird blind übernommen
- LLM trifft eine Entscheidung, die regelbasiert geprüft werden sollte

Validatoren

1. Static Validator

Prüft:

- JSON-Struktur
- Node-Typen
- Connections
- Erreichbarkeit
- tote Pfade

Stärken:

- stark bei technischen und strukturellen Fehlern

Schwächen:

- schwach bei fachlicher Semantik

2. Rule-Based Business Validator

Prüft:

- Muss-Schritte
- Reihenfolge
- Branches

- Parallelitaet
- Soll-Ist-Abgleich mit Referenzspezifikation

Staerken:

- stark bei modellierbarer Business-Logik

Schwaechen:

- begrenzt bei offenen Bedeutungen und Sprachvarianten

3. LLM-as-a-Judge

Input:

- fachliche Beschreibung
- Workflow-JSON oder abstrahierte Workflow-Sicht

Output:

- **valid** oder **invalid**
- Begrueendung
- optional Fehlerkategorie und Confidence

Staerken:

- besser bei semantischen Widerspruechen

Schwaechen:

- potenziell instabil bei subtilen Strukturdetails
- teurer

4. Hybrid

Kombiniert:

- statische und regelbasierte Vorpruefung
- LLM-basierte Restpruefung fuer semantische Abweichungen

Staerken:

- voraussichtlich insgesamt am robustesten

Schwaechen:

- hoehere Komplexitaet
- hoehere Kosten

Zusaetzliche Forschungsfrage: Komplexitaet

Neben der Gesamtleistung wird untersucht, ob die Validierungsqualitaet mit steigender Workflow-Komplexitaet sinkt.

Workflow-Komplexitaet wird operationalisiert ueber:

- Anzahl der Nodes
- Anzahl der Edges
- Anzahl der Branches
- Anzahl der Merges
- Tiefe des Kontrollflusses
- Anzahl externer Integrationen
- Anzahl von LLM-Knoten

Auswertung:

- Leistung pro Komplexitaetsstufe
- Fehler, die erst ab Komplexitaet **x** uebers ehen werden
- Unterschiede zwischen klassischen und LLM-haltigen Workflows

Zusaetzliche Forschungsfrage: LLM-Aufgaben im Workflow

Ein eigener Teil der Evaluation betrachtet LLM-Aufgaben innerhalb eines Workflows.

Dabei wird nicht versucht, die vollstaendige inhaltliche Wahrheit der LLM-Ausgabe zu validieren. Stattdessen wird geprueft:

- ob der LLM-Knoten fachlich an der richtigen Stelle sitzt
- ob sein Zweck zur Beschreibung passt
- ob seine Ausgabe weiter validiert wird
- ob kritische Entscheidungen nicht blind auf dem LLM beruhen
- ob die Ausgabe in den richtigen Branch fliesst

Hier werden zwei Ansaetze verglichen:

- **LLM-as-a-Judge** fuer den gesamten LLM-Teil
- **regelbasierte Approximation** ohne Judge-Modell

Die regelbasierte Approximation kann auf Techniken wie in [validate_n8n_example.ps1](#) beruhen:

- Keyword-Matching
- Stopwort-Filter
- Start-, End- und Branch-Erkennung
- Pruefung auf erwartete Knotenbezeichnungen
- Abgleich von Parallelitaet und Alternativen

Damit laesst sich untersuchen, ob bei bestimmten LLM-Aufgaben eine relativ einfache, erklaeerbare Validierung bereits ausreichend ist und wo ein echtes Judge-Modell weiterhin notwendig bleibt.

Testablauf

Fuer jedes Testobjekt:

1. Referenzbeschreibung und Referenzspezifikation laden
2. Validator auf den Workflow anwenden

3. Ausgabe speichern:

- **valid** oder **invalid**
- erkannte Fehler
- Confidence, falls verfuegbar
- Laufzeit
- Tokenkosten, falls LLM verwendet wird

4. Ergebnis mit Ground Truth vergleichen

Metriken

Pflichtmetriken:

- Accuracy
- Precision
- Recall
- F1
- False Positive Rate
- False Negative Rate

Zusaetzliche Auswertungen:

- pro Fehlerklasse
- pro Komplexitaetsstufe
- fuer Workflows mit und ohne LLM-Node
- durchschnittliche Laufzeit
- durchschnittliche Tokenkosten

Moegliche Ergebnisdarstellungen

Damit die Evaluation spaeter nicht nur aus Text besteht, sollten die Resultate mit mehreren komplementaeren Darstellungen aufbereitet werden.

Tabellen

- Tabelle mit Gesamtwerten je Validator: Accuracy, Precision, Recall, F1
- Tabelle je Fehlerklasse: Struktur, Kontrollfluss, Semantik, LLM-bezogen
- Tabelle je Komplexitaetsstufe: niedrig, mittel, hoch
- Tabelle fuer Workflows mit und ohne LLM-Knoten
- Tabelle fuer durchschnittliche Laufzeit und durchschnittliche Tokenkosten

Balkendiagramme

- Balkendiagramm: **F1** pro Validator
- Balkendiagramm: **Recall** pro Fehlerklasse und Validator
- Balkendiagramm: **False Negative Rate** fuer komplexe Workflows
- Balkendiagramm: Vergleich von LLM-Workflows gegen klassische Workflows

Linien- oder Kurvendiagramme

- Linie: Leistung eines Validators ueber steigende Node-Anzahl
- Linie: Leistung ueber steigende Branch-Anzahl
- Linie: Recall in Abhaengigkeit von der Workflow-Tiefe
- Linie: Fehlerrate ab Komplexitaet x

Heatmaps

- Heatmap: Validatoren mal Fehlerklassen
- Heatmap: Validatoren mal Komplexitaetsstufen
- Heatmap: LLM-Node-Typen mal Erkennungsrate

Boxplots

- Boxplot: Laufzeit pro Validator
- Boxplot: Tokenkosten pro LLM-basiertem Validator
- Boxplot: Streuung der Leistung ueber verschiedene Domains

Konfusionsmatrizen

- Konfusionsmatrix je Validator fuer **valid** gegen **invalid**
- getrennt fuer Gesamtmenge und fuer LLM-Teilmenge

Fallbasierte Visualisierungen

- kleine Workflow-Abbildung fuer ausgewaehlte gute und schlechte Beispiele
- dazu Tabelle mit:
- Ground Truth
- Urteil des Validators
- erkannte Fehler
- uebersehene Fehler

Moegliche Ergebnislisten im Ergebniskapitel

Im Ergebnisteil der Arbeit koennten die Resultate in etwa in folgender Reihenfolge praesentiert werden:

1. Gesamtvergleich aller Validatoren

- Welcher Validator erzielt die beste Gesamtleistung
- Welche Unterschiede sind statistisch oder praktisch relevant

2. Ergebnisse nach Fehlerklasse

- Wer erkennt Strukturfehler am besten
- Wer erkennt Kontrollflussfehler am besten
- Wer erkennt semantische Fehler am besten
- Wie schneiden die Validatoren bei LLM-bezogenen Fehlern ab

3. Ergebnisse nach Komplexitaet

- Ab welcher Node-Anzahl sinkt die Leistung
- Ab welcher Branching-Tiefe werden Fehler haeufig uebersehen

- Welche Methode ist am robustesten bei komplexen Workflows

4. Ergebnisse fuer LLM-Workflows

- Wie stark sinkt die Validierungsqualitaet bei LLM-Knoten
- Wo ist die regelbasierte Approximation ausreichend
- Wo braucht man klar ein Judge-Modell

5. Kosten-Nutzen-Betrachtung

- Wie stark verbessert der Hybridansatz die Leistung
- Rechtfertigt der Leistungsgewinn die zusaetzlichen Tokenkosten
- Welche Methode ist fuer praktische Nutzung am effizientesten

6. Qualitative Fehleranalyse

- typische uebersehene Fehler je Validator
- typische Fehlalarme je Validator
- konkrete Beispiele fuer Grenzfaelle

Erwartete, nicht-triviale Ergebnisse

Es wird nicht erwartet, dass ein einzelner Validator in allen Dimensionen dominiert.

Wahrscheinliche Muster:

- Static Validator ist am besten bei Strukturfehlern
- Rule-Based Validator ist gut bei klaren Invarianten
- LLM-as-a-Judge ist besser bei semantischen Abweichungen
- Hybrid ist insgesamt am robustesten, aber teurer
- Bei LLM-Aufgaben sinkt die Zuverlaessigkeit aller Verfahren, aber nicht gleich stark

Genau diese differenzierten Unterschiede machen die Evaluation wissenschaftlich interessant.

Fazit des Testaufbaus

Dieser Aufbau erlaubt:

- einen fairen Vergleich mehrerer Validierungsansaetze
- eine Untersuchung von Fehlererkennung in Abhaengigkeit von Komplexitaet
- eine gezielte Analyse, ob LLM-Aufgaben im Workflow auch ohne Judge-Modell teilweise validierbar sind

Damit entstehen voraussichtlich heterogene Ergebnisse statt einer trivialen Aussage wie **alles validierbar** oder **gar nichts validierbar**.